

JWT (JSON Web Token) é uma das formas mais utilizadas para autenticação em APIs REST porque é **stateless**, ou seja, o servidor não precisa armazenar sessão.

Como funciona o fluxo

1. Usuário faz login
↓
 2. API valida usuário e senha
↓
 3. API gera um JWT
↓
 4. JWT é enviado para o cliente
↓
 5. Cliente envia o token em todas as requisições
Authorization: Bearer TOKEN
↓
 6. Spring Security valida o token
↓
 7. Se válido → acesso liberado
-

Dependências Maven

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-security</artifactId>  
</dependency>
```

```
<dependency>  
  <groupId>io.jsonwebtoken</groupId>  
  <artifactId>jjwt-api</artifactId>  
  <version>0.12.5</version>  
</dependency>
```

```
<dependency>  
  <groupId>io.jsonwebtoken</groupId>  
  <artifactId>jjwt-impl</artifactId>  
  <version>0.12.5</version>  
  <scope>runtime</scope>  
</dependency>
```

```
<dependency>  
  <groupId>io.jsonwebtoken</groupId>
```

```
<artifactId>jjwt-jackson</artifactId>
<version>0.12.5</version>
<scope>runtime</scope>
</dependency>
```

Entidade Usuario

```
@Entity
public class Usuario {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String username;

    private String password;

    private String role;

    // getters e setters
}
```

Repository

```
@Repository
public interface UsuarioRepository
    extends JpaRepository<Usuario, Long> {

    Optional<Usuario> findByUsername(String username);
}
```

UserDetailsService

O Spring Security usa essa classe para buscar usuários.

```
@Service
public class UsuarioDetailsService
    implements UserDetailsService {
```

```

@Autowired
private UsuarioRepository repository;

@Override
public UserDetails loadUserByUsername(String username)
    throws UsernameNotFoundException {

    Usuario usuario = repository.findByUsername(username)
        .orElseThrow(() ->
            new UsernameNotFoundException("Usuário não encontrado"));

    return User.builder()
        .username(usuario.getUsername())
        .password(usuario.getPassword())
        .roles(usuario.getRole())
        .build();
}
}

```

Classe JWT Service

Responsável por gerar e validar tokens.

```

@Service
public class JwtService {

    private final String SECRET =
        "12345678901234567890123456789012";

    private SecretKey getKey() {
        return Keys.hmacShaKeyFor(
            SECRET.getBytes(StandardCharsets.UTF_8));
    }

    public String gerarToken(String username) {

        return Jwts.builder()
            .subject(username)
            .issuedAt(new Date())
            .expiration(
                new Date(System.currentTimeMillis()

```

```

        + 1000 * 60 * 60))
        .signWith(getKey())
        .compact();
    }

    public String extrairUsuario(String token) {

        return Jwts.parser()
            .verifyWith(getKey())
            .build()
            .parseSignedClaims(token)
            .getPayload()
            .getSubject();
    }

    public boolean tokenValido(String token) {

        try {
            Jwts.parser()
                .verifyWith(getKey())
                .build()
                .parseSignedClaims(token);

            return true;

        } catch (Exception e) {
            return false;
        }
    }
}

```

DTO Login

```

public class LoginRequest {

    private String username;
    private String password;

    // getters e setters
}

```

DTO Resposta

```
public class LoginResponse {  
  
    private String token;  
  
    public LoginResponse(String token) {  
        this.token = token;  
    }  
  
    public String getToken() {  
        return token;  
    }  
}
```

Controller de Login

```
@RestController  
@RequestMapping("/auth")  
public class AuthController {  
  
    @Autowired  
    private AuthenticationManager authManager;  
  
    @Autowired  
    private JwtService jwtService;  
  
    @PostMapping("/login")  
    public LoginResponse login(  
        @RequestBody LoginRequest request) {  
  
        Authentication auth =  
            authManager.authenticate(  
                new UsernamePasswordAuthenticationToken(  
                    request.getUsername(),  
                    request.getPassword());  
  
        String token =  
            jwtService.gerarToken(auth.getName());
```

```
        return new LoginResponse(token);
    }
}
```

Filtro JWT

Esse filtro intercepta todas as requisições.

@Component

```
public class JwtFilter extends OncePerRequestFilter {
```

```
    @Autowired
```

```
    private JwtService jwtService;
```

```
    @Autowired
```

```
    private UsuarioDetailsService userDetailsService;
```

```
    @Override
```

```
    protected void doFilterInternal(
        HttpServletRequest request,
        HttpServletResponse response,
        FilterChain filterChain)
        throws ServletException, IOException {
```

```
        String authHeader =
            request.getHeader("Authorization");
```

```
        if (authHeader == null ||
            !authHeader.startsWith("Bearer ")) {
```

```
            filterChain.doFilter(request, response);
            return;
        }
```

```
        String token =
            authHeader.substring(7);
```

```
        String username =
            jwtService.extrairUsuario(token);
```

```
        if (username != null &&
```

```

SecurityContextHolder.getContext()
    .getAuthentication() == null) {

    UserDetails user =
        userDetailsService
            .loadUserByUsername(username);

    UsernamePasswordAuthenticationToken auth =
        new UsernamePasswordAuthenticationToken(
            user,
            null,
            user.getAuthorities());

    SecurityContextHolder.getContext()
        .setAuthentication(auth);
}

filterChain.doFilter(request, response);
}
}

```

Configuração Security

```

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Autowired
    private JwtFilter jwtFilter;

    @Autowired
    private UsuarioDetailsService userDetailsService;

    @Bean
    PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    AuthenticationManager authenticationManager(

```

```

        AuthenticationConfiguration config)
        throws Exception {

    return config.getAuthenticationManager();
}

@Bean
SecurityFilterChain securityFilterChain(
    HttpSecurity http) throws Exception {

    return http
        .csrf(csrf -> csrf.disable())

        .sessionManagement(sm ->
            sm.sessionCreationPolicy(
                SessionCreationPolicy.STATELESS))

        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/auth/**")
            .permitAll()
            .requestMatchers("/admin/**")
            .hasRole("ADMIN")
            .anyRequest()
            .authenticated())

        .addFilterBefore(
            jwtFilter,
            UsernamePasswordAuthenticationFilter.class)

        .build();
}
}

```

Cadastro de Usuário

```

@PostMapping("/usuarios")
public Usuario cadastrar(
    @RequestBody Usuario usuario) {

    usuario.setPassword(

```

```
        passwordEncoder.encode(
            usuario.getPassword());

    return repository.save(usuario);
}
```

Exemplo de uso

Criar usuário

POST /usuarios

```
{
  "username":"admin",
  "password":"123456",
  "role":"ADMIN"
}
```

Senha será gravada criptografada:

\$2a\$10\$N8...

Login

POST /auth/login

```
{
  "username":"admin",
  "password":"123456"
}
```

Resposta:

```
{
  "token":"eyJhbGciOiJIUzI1NiJ9..."
}
```

Consumir endpoint protegido

GET /produtos

Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...

Controle por perfil

Você pode proteger endpoints com:

```
@PreAuthorize("hasRole('ADMIN')")
@PostMapping
public Produto salvar() {
}
```

ou

```
@PreAuthorize("hasAnyRole('ADMIN','GERENTE')")
```

Lembre-se de habilitar:

```
@Configuration
@EnableMethodSecurity
public class SecurityConfig {
}
```

Problema comum que gera erro 401

Se você consegue cadastrar usuários mas recebe 401 no login, normalmente é um destes problemas:

1. Senha foi salva sem BCryptPasswordEncoder.
2. Senha está sendo criptografada duas vezes.
3. UserDetailsService não encontra o usuário.
4. Usuário foi salvo com username diferente do informado no login.
5. O AuthenticationManager não está usando seu UserDetailsService.
6. O token JWT não está sendo enviado como: